CLASE 4 - Sistemas Digitales

Semana 4: Puertos, registros y GPIO

1. Puertos y registros en AVR

Un puerto agrupa varios pines de entrada y salida. En el AVR ATmega328P, los puertos B, C y D tienen hasta 8 bits asociados, aunque no todos sus pines están disponibles físicamente en todos los encapsulados.

Cada bit de un registro controla o representa el estado de un pin del puerto.

Registros principales del puerto B

Registro	¿Para qué sirve?	Ejemplo con el bit 1
`DDRB`	Define si cada pin funciona como entrada o salida.	`1`: salida. `0`: entrada.
`PORTB`	Escribe el valor de salida o activa la resistencia pull-up si el pin es entrada.	Salida: coloca un nivel alto. Entrada: activa la pull-up.
`PINB`	Permite leer el estado lógico de los pines.	`1`: el pin se encuentra en nivel alto. `0`: está en nivel bajo.

Ejemplo: si se configura el bit 1 de `DDRB` en `1`, el pin correspondiente del puerto B funciona como salida.

Resistencia pull-up

La resistencia pull-up mantiene un pin de entrada en un nivel lógico alto cuando no está conectado a una señal que lo lleve a nivel bajo.

- Sin pulsar el botón: se obtiene un `1` lógico.
- Al pulsar un botón conectado a GND: se obtiene un `0` lógico.

En AVR se activa colocando el bit correspondiente de `PORTB` en `1`, siempre que el pin esté configurado como entrada.

Alta impedancia (Hi-Z)

Es un estado en el que el pin presenta una impedancia muy alta y queda prácticamente desconectado eléctricamente del circuito interno de salida.

- Se utiliza cuando el pin funciona como entrada.
- Permite que una señal externa determine su nivel lógico.

- Si no tiene una señal definida ni una resistencia pull-up o pull-down, puede quedar flotante y producir lecturas inestables.

2. Lectura y configuración de pines

Los periféricos del microcontrolador se controlan mediante registros que tienen direcciones dentro del espacio de memoria o de registros de entrada/salida.

En AVR:

- `DDR`: selecciona si el pin será entrada o salida.
- `PORT`: establece el valor de salida o activa la resistencia pull-up.
- `PIN`: permite leer el estado lógico del pin.

Ejemplos de operaciones

1. Conversión hexadecimal a binario

El valor `0x81` representa:

```
Hexadecimal: 0x81
Binario:     10000001
```

Los bits 7 y 0 están en `1`; los demás están en `0`.

2. Activar el bit 4 de PORTB

```
PORTB |= (1 << 4);
```

La expresión `1 << 4` desplaza el número 1 cuatro posiciones a la izquierda. Luego, el operador OR (`|`) activa el bit 4 sin modificar los demás bits.

3. Leer el pin D2 con un botón conectado a GND

```
DDRD &= ~(1 << 2);
PORTD |= (1 << 2);

if (PIND & (1 << 2)) {
    // El botón no está pulsado: nivel alto
} else {
    // El botón está pulsado: nivel bajo
}
```

- `DDRD &= ~(1 << 2)`: configura D2 como entrada.
- `PORTD |= (1 << 2)`: activa la resistencia pull-up.
- `PIND`: permite leer el estado real del pin.

Como el botón está conectado a GND, al pulsarlo el pin pasa a nivel bajo (`0`). Por tanto, la condición del `if` será falsa y se ejecutará el `else` .

3. Del AVR al ESP32

Un periférico es un circuito interno del microcontrolador que cumple una función específica, como controlar entradas, salidas, temporizadores o comunicaciones.

Un pin es una conexión física que permite comunicar el microcontrolador con componentes externos. Puede funcionar como entrada o salida y se controla mediante registros.

Registros y flip-flops

Los registros son espacios de almacenamiento internos que permiten controlar o consultar el funcionamiento del microcontrolador. Están implementados mediante circuitos digitales, generalmente construidos a partir de flip-flops.

Al escribir determinados bits en los registros permitidos, se modifica el comportamiento de los periféricos y de los pines.

- El pin es una conexión física.
- Los registros permiten configurar y controlar su funcionamiento.
- No se puede escribir en cualquier dirección: cada registro tiene una función específica.

4. Estructura de la placa ESP32 DevKit V1

La placa DevKit V1 incorpora un módulo ESP32-WROOM-32 y circuitos adicionales que permiten alimentarlo y programarlo desde una computadora.

Componentes principales

Conexión USB:

La conexión USB proporciona alimentación de 5 V y permite comunicarse con la computadora.

Conversor USB a serial:

Chips como el CP2102 o CH340 convierten la comunicación USB de la computadora en comunicación serial que puede utilizar el ESP32.

Regulador de voltaje:

Convierte los 5 V provenientes del USB a 3.3 V para alimentar el módulo ESP32.

Módulo ESP32-WROOM-32:

Contiene los siguientes componentes:

- **Chip ESP32:** procesador de dos núcleos en la versión habitual del WROOM-32.
- **Memoria Flash:** almacena el programa y otros datos. En la versión común del módulo se encuentran 4 MB.
- **Antena:** permite la comunicación inalámbrica mediante Wi-Fi y Bluetooth.

Los pines GPIO 6 al 11 están asociados normalmente a la comunicación con la memoria Flash del módulo. Por eso, no deben utilizarse como GPIO de propósito general.

Botones de la placa:

- `EN` : permite reiniciar el ESP32.
- `BOOT` : permite entrar en el modo de descarga de programas. Está relacionado con GPIO 0.

La placa DevKit V1 puede encontrarse en versiones con diferentes cantidades de pines, por ejemplo, 30 pines.

5. Comunicación y funcionamiento de los pines

Los microcontroladores utilizan diferentes formas de comunicación, entre ellas la comunicación serial, que transmite información mediante una secuencia de bits.

La memoria Flash almacena el programa que ejecuta el microcontrolador.

Un pin es el puerto físico que conecta el circuito interno del microcontrolador con los componentes externos.

Configuración de un GPIO en el ESP32

Para configurar un pin se deben considerar tres aspectos:

1. **Elegir su función:** seleccionar si será entrada, salida o una función alternativa. Esto se relaciona con el multiplexor de funciones (MUX).
2. **Conectar la señal:** establecer la ruta de la señal mediante la matriz de interconexión GPIO.
3. **Habilitar la salida:** configurar el pin para que pueda generar una señal cuando se necesite.

En Arduino, la función `pinMode()` realiza la configuración necesaria para establecer un pin como entrada o salida, sin tener que configurar manualmente cada uno de esos elementos.

```
pinMode(2, OUTPUT);  // Configura GPIO 2 como salida
pinMode(4, INPUT);   // Configura GPIO 4 como entrada
```

GPIO

GPIO significa *General Purpose Input/Output* (entrada/salida de propósito general).

Permite:

- Colocar un pin en nivel lógico `0`.
- Colocar un pin en nivel lógico `1`.
- Leer el estado lógico de un pin.

En el ESP32, los niveles lógicos trabajan normalmente con 0 V para el nivel bajo y 3.3 V para el nivel alto. Sus GPIO no deben recibir directamente señales de 5 V.

6. Registros W1TS y W1TC del ESP32

El ESP32 dispone de registros que permiten modificar directamente el estado de sus salidas GPIO.

W1TS: Write One to Set

Permite activar bits de salida.

- Escribir `1`: establece el bit en `1`.
- Escribir `0`: mantiene el estado anterior.

W1TC: Write One to Clear

Permite limpiar o desactivar bits de salida.

- Escribir `1`: establece el bit en `0`.
- Escribir `0`: mantiene el estado anterior.

La ventaja de estos registros es que permiten modificar bits específicos sin tener que leer y reescribir todo el registro de salida.

7. Riesgo de leer, modificar y escribir

Una operación de lectura, modificación y escritura consiste en:

1. **Leer:** obtener el valor actual del registro.
2. **Modificar:** cambiar el bit o los bits necesarios.
3. **Escribir:** guardar el nuevo valor en el registro.

El problema aparece cuando otra operación, por ejemplo una interrupción, modifica el mismo registro entre la lectura y la escritura.

Esto puede provocar que se pierda una modificación o que el registro termine con un valor incorrecto.

Por eso, los registros W1TS y W1TC del ESP32 son útiles: permiten establecer o limpiar bits directamente, sin realizar una operación completa de lectura y modificación.